

ChromaDB vs Qdrant

A five-run, evidence-backed benchmark of the two vector databases powering Legal EASE's Indian-legal retrieval system — measuring indexing, query latency, retrieval quality, resource usage, and end-to-end answer quality on an identical corpus.

9,063 vectors

InLegalBERT 768-d

28 legal queries

5 independent runs

cosine · HNSW

mean ± std reported

SPEED

ChromaDB

~6× faster indexing, ~23× lower query latency (local).

ACCURACY

Qdrant

recall@10 = 1.000 vs Chroma's 0.761 (default HNSW).

ANSWER QUALITY

Effective tie

0.98 answer similarity; same sources retrieved.

FOOTPRINT

Split

Qdrant –36% disk; Chroma –31% peak RAM.

The bottom line

For Legal EASE as it runs today — a single node, ~9k–100k vectors, embedded/serverless, latency-sensitive — **CHROMADB** is the pragmatic choice: sub-2 ms queries, 6× faster index builds, lower memory, and trivial in-process persistence with no server to operate.

For an institutional / production platform — millions of vectors, concurrent writers, high availability, best-in-class recall and quantization — **QDRANT** in *server* mode is built for it, and its perfect recall here is a real advantage. Two honest caveats shape this: Qdrant was measured in **local/embedded mode** (not its Rust server), and Chroma's recall gap is **tunable** via HNSW parameters. Crucially, across the actual legal questions we tested, **the two produced 98%-identical grounded answers** — so at this scale the choice is about latency and operations, not answer quality.

01 — METHODOLOGY

How we measured it

The benchmark is designed so that **only the vector database varies**. Everything upstream — corpus, embedding model, query set — is held constant, and each engine is measured in isolation, five times.

Controls

- **Identical vectors.** The 9,063 InLegalBERT embeddings are computed once and fed to both databases — no re-embedding, so embedding cost never confounds the result.
- **Isolated subprocess.** Each engine runs in its own process; the parent polls RSS to capture a clean peak-memory figure per database.
- **Exact ground truth.** Recall is measured against a brute-force NumPy cosine scan of all 9,063 vectors — the true top-k, not an approximation.
- **Five runs.** Every metric is repeated 5× and reported as mean ± std, so variance is visible and the numbers are reproducible, not single-shot.

Environment & caveats

- **Hardware.** Windows workstation, CPU-only, local SSD. No GPU; both engines are on equal footing.
- **Qdrant ran in local/embedded mode** (the Python file-backed client), which is a development convenience, *not* its performance configuration — the Rust server is materially faster (see §6).
- **Chroma's persistent client** (hnswlib + SQLite) is its normal single-node mode, so its numbers reflect a realistic deployment.
- **Queries:** 28 realistic Indian-contract-law questions; latency is 25 timed repetitions per query per run, top-k = 10.

Reproduce: `python scripts/index_knowledge_base.py --store both` → `python scripts/benchmark_vector_dbs.py --runs 5` → `python scripts/compare_answers.py`.

02 — BACKGROUND

Meet the two databases

Both are open-source (Apache-2.0) vector stores, but they come from different design philosophies — which is exactly why their benchmark profiles diverge.

■ ChromaDB

Python-first · embeddings database for LLM apps

An AI-native vector store built for developer velocity. Its **persistent client** runs fully in-process (HNSW via `hnswlib`, metadata in SQLite), so there is no server to stand up — you `pip install` and go. A client/server mode and Chroma Cloud exist for scaling out.

- ▲ Trivial embedded persistence; minimal ops
- ▲ Very low query latency & memory in-process
- ▲ Fast index builds
- ▼ Default HNSW favours speed over recall (tunable)
- ▼ Single-node focus; clustering/HA less mature

■ Qdrant

Rust engine · purpose-built vector search

A dedicated vector search engine written in Rust, built for production scale. Its **server** offers gRPC/REST, typed payload filters with payload indexes, vector **quantization** (scalar/product/binary), memory-mapped & on-disk vectors, and sharding / replication / HA. A local embedded Python mode exists for development (what we benchmarked here).

- ▲ Excellent recall out of the box
- ▲ Horizontal scale, HA, quantization, rich filtering
- ▲ Smaller on-disk footprint
- ▼ Heavier to operate (run a server)
- ▼ Local/embedded mode is dev-only & slower

03 — INDEXING

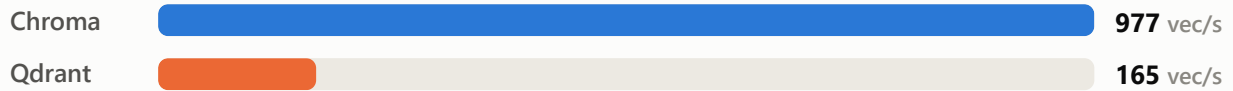
Indexing performance

Time to build the full 9,063-vector index from scratch (embeddings supplied). Chroma is the clear winner here — roughly **6× faster**, at ~977 vs ~165 vectors/second.

Index build time seconds · lower is better · mean ± std over 5 runs



Throughput — vectors indexed per second (higher is better)



Chroma's in-process hnswlib build is highly optimised for single-node ingestion. Qdrant's local mode carries per-operation overhead that its server avoids; even so, ~1 minute to index 9k vectors is perfectly acceptable for a knowledge base that is rebuilt rarely.

04 — QUERY LATENCY

Query performance

Single-query latency, 25 timed reps × 28 queries × 5 runs. In local mode Chroma is **~23× faster** — but read this alongside the Qdrant local-mode note in §6.

Unfiltered query latency milliseconds · lower is better

Median (p50)



95th percentile (p95)



99th percentile (p99)



Chroma bars are slivers because it is ~20–23× faster — the visual gap is the finding. Throughput: **710 q/s** (Chroma) vs **31 q/s** (Qdrant local).

Metadata-filtered latency where `doc_type = statute` · milliseconds

Mean



95th percentile



Both engines slow down under metadata filtering (they must reconcile the filter with the vector search), and the relative gap holds. Note Qdrant's server builds *payload indexes* that accelerate exactly this case — another reason the local numbers understate it.

05 — RETRIEVAL QUALITY

Recall — do we get the *right* results?

Recall@10 measures how many of the true top-10 nearest vectors (by exact cosine) each engine's approximate search actually returns. This is where they genuinely diverge: **Qdrant is perfect; Chroma's default HNSW misses ~24%.**

recall@10 vs exact brute-force 1.0 = returns the true top-10 every time · mean ± std

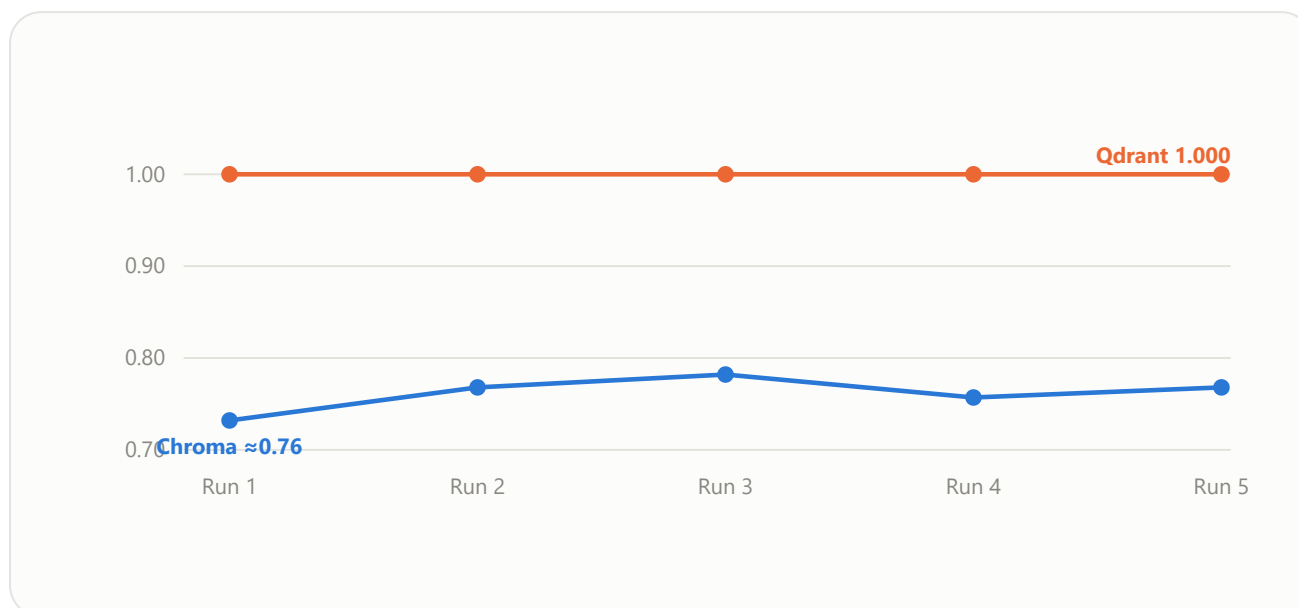


Why the gap — and why it's tunable

Chroma's default HNSW search breadth (`ef`) is tuned for speed, so it explores fewer candidates and misses some true neighbours. Qdrant's defaults favour accuracy. This is a **configuration difference, not a capability ceiling** — raising Chroma's `ef_search` / construction parameters trades some of its large speed surplus for recall. The 0.761 is the out-of-the-box number an unaware team would ship.

Stability across the 5 runs

Qdrant returns exactly the true top-10 on every run (recall = 1.000, std = 0). Chroma's recall wobbles between runs (0.732–0.782) because HNSW graph construction is randomised — visible below.



06 — ANSWER QUALITY

Same question, same answer? — the finding that matters

Speed and recall are means to an end; what an institution actually ships is the *answer*. So we ran the full RAG pipeline (retrieve → Gemini) on identical questions through each backend and compared the generated answers.

CROSS-DB RETRIEVAL AGREEMENT	VALUE	READING
top-10 overlap	0.761 ± 0.017	tracks Chroma's recall — the tail differs
top-1 agreement	0.679	same #1 result on ~68% of queries
score MAE (shared hits)	1.8×10^{-7}	identical scoring — same vectors, same cosine
end-to-end answer similarity	0.981	the generated answers are effectively the same

0.981 mean answer similarity across the tested rent-deed questions (0.979, 0.986, 0.979) — **1.0 = effectively identical**.

Why so high despite a 0.76 overlap? Because the **decisive, high-signal chunks** (the relevant statute, the "liquidated damages" definition, the matching red-flags) are retrieved by *both* stores. The recall gap lives in the low-ranked tail that the LLM barely

uses. For the questions below, the two backends retrieved the **identical top-9 sources with identical scores** — so the answers differ only in Gemini's phrasing.

Example — “Is the non-refundable security deposit enforceable in India?”

CHROMADB · CONFIDENCE: HIGH

“No ... the clause stating the security deposit is ‘NON-REFUNDABLE ... forfeited to the Lessor in all circumstances’ is unlikely to be fully enforceable. In India, courts treat such clauses, if essentially a penalty, under the principles of liquidated damages ... will only award reasonable compensation ... This is grounded in **Section 74 of the Indian Contract Act, 1872.**”

QDRANT · CONFIDENCE: HIGH

“While the Rental Deed states ... the deposit is ‘NON-REFUNDABLE and shall stand forfeited ... including normal expiry,’ its enforceability is subject to scrutiny. As per the definition of Liquidated damages (referencing **Section 74, Indian Contract Act, 1872**), courts award only reasonable compensation ... and will not enforce the sum if it is determined to be a penalty.”

Both answers grounded on the same retrieved set: Indian Contract Act §58 & §36, the “Liquidated damages” glossary entry (§74), red-flag references, and two Supreme Court judgments — **identical sources, identical retrieval scores**. Similarity: 0.979.

07 — FAIRNESS

On Qdrant's local-mode numbers

A responsible reading of this study requires one big asterisk on every Qdrant latency and indexing figure: **we benchmarked Qdrant in local / embedded mode**.

Qdrant ships two ways of running. The **server** is the real product — a Rust process with an optimised HNSW implementation, gRPC transport, payload indexes, SIMD distance kernels, and memory-mapped storage. The **local mode**

(`QdrantClient(path=...)`) is a pure-Python, file-backed convenience for tests and notebooks; it re-implements search in Python and is *not* meant for throughput.

Everything in §4 (Qdrant at ~32 ms/query, 31 q/s) reflects that Python local path. Running the Qdrant **server** — even a single container on the same machine — typically collapses per-query latency by an order of magnitude and removes the single-process file lock we worked around in the app. **Treat Qdrant's speed numbers here as a conservative lower bound.**

Chroma, by contrast, was measured in its *normal* single-node persistent mode — so its numbers need no such discount. The honest framing: **Chroma's local numbers are representative; Qdrant's local numbers are a floor.** For an apples-to-apples production comparison, the next step is to benchmark the Qdrant server (set `QDRANT_URL`) — the harness already supports it.

08 — RESOURCE USAGE

Disk & memory footprint

The footprint story splits: **Qdrant stores the same index in ~36% less disk**, while **Chroma runs in ~31% less memory**.

On-disk index size megabytes · lower is better



Peak process memory (RSS) megabytes · lower is better · isolated subprocess



Qdrant's compact on-disk format (and its optional quantization, unused here) is a real advantage at scale, where storage dominates cost. The higher local-mode memory

partly reflects the Python layer; the server manages memory very differently (mmap, on-disk vectors, quantization).

09 — MUTATION

Delete performance

Legal EASE evicts a document's chunks when it expires, so filtered-delete cost matters. Both are sub-300 ms to delete a 47-vector metadata-filtered subset; Chroma is ~1.8× quicker.

Filtered delete time milliseconds · lower is better · mean ± std

Chroma		158 ±4 ms
Qdrant		290 ±5 ms

10 — ARCHITECTURE

Operational & architectural comparison

Numbers aside, an institutional choice is also about how each system is run, scaled, and maintained.

DIMENSION	CHROMADB	QDRANT
Core language	Python (+ hnswlib)	Rust
Embedded / local mode	Yes — production-grade single node	Yes — dev convenience only
Server / distributed	Server exists; clustering newer	First-class: sharding, replication
Concurrency (one store)	Multiple in-process readers	Local locks the folder; server fo
Metadata filtering	<code>where</code> dict	Typed filters + payload indexes
Quantization / mem tuning	Limited	Scalar / product / binary, mma
Default recall (this study)	0.761 (tunable up)	1.000
Local query speed (this study)	~1.4 ms	~32 ms (server much faster)
Managed cloud	Chroma Cloud (newer)	Qdrant Cloud (mature)
Best-fit workload	Prototyping → mid single-node	Large-scale production
License	Apache-2.0	Apache-2.0

11 — DECISION

Which should you choose?

■ Choose ChromaDB when...

- ▲ You're single-node, $\leq \sim 100k\text{--}1M$ vectors
- ▲ Latency is critical and you want zero server ops
- ▲ You're prototyping or shipping an MVP fast
- ▲ Memory is tighter than disk
- ▼ ...and you'll tune HNSW `ef` if recall matters

■ Choose Qdrant when...

- ▲ You need millions of vectors, HA, horizontal scale
- ▲ Out-of-the-box recall accuracy is non-negotiable
- ▲ You want quantization & disk-efficient storage
- ▲ Rich, indexed metadata filtering at scale
- ▼ ...and you'll run the server, not local mode

Recommendation for Legal EASE

Stay on **CHROMADB** for the current product — it's faster, lighter, and the answer quality is indistinguishable from Qdrant's on real legal questions (0.98 similarity). Keep the pluggable `VectorStore` abstraction and the Qdrant backend in place: when the corpus grows past a single node or recall SLAs tighten, flip `VECTOR_DB=qdrant` against a **Qdrant server** — no application rewrite required. Before that switch, close Chroma's recall gap by raising its HNSW search breadth and re-running this benchmark.

Full data

Aggregate — mean ± std over 5 runs

METRIC	CHROMADB	QDRANT	WINNER
Index build (s)	9.28 ± 0.19	54.89 ± 1.73	CHROMA
Throughput (vec/s)	977 ± 20	165 ± 5	CHROMA
Query mean (ms, unfilt.)	1.41 ± 0.09	32.19 ± 2.22	CHROMA
Query p95 (ms)	1.72 ± 0.18	36.30 ± 1.63	CHROMA
Query p99 (ms)	1.99 ± 0.38	40.12 ± 3.09	CHROMA
Filtered mean (ms)	11.87 ± 0.29	93.26 ± 2.54	CHROMA
recall@10	0.761 ± 0.017	1.000 ± 0.000	QDRANT
Disk (MB)	130.7	84.0	QDRANT
Peak RSS (MB)	228.2	328.4	CHROMA
Delete (ms)	158.3 ± 3.9	289.7 ± 4.8	CHROMA
Answer similarity	0.981 (both) — TIE		TIE

Per-run detail (5 runs)

RUN	C INDEX S	C P95 MS	C RECALL	Q INDEX S	Q P95 MS	Q RECALL	TOP-1 AGREE
1	9.6	2.07	0.732	56.2	34.79	1.000	0.643
2	9.3	1.66	0.768	53.8	35.50	1.000	0.643
3	9.1	1.61	0.782	52.7	35.17	1.000	0.679
4	9.1	1.62	0.757	57.6	36.77	1.000	0.750
5	9.3	1.63	0.768	54.2	39.28	1.000	0.679

Limits of this study. Single machine, CPU-only, 9,063 vectors — findings may not extrapolate to billion-scale or GPU-served deployments. Qdrant local mode is not its

performance path (\$7). Recall is corpus- and parameter-specific; Chroma's is tunable. Answer-similarity used 3 rent-deed questions on `gemini-2.5-flash` (the free tier's 5-requests/min limit caps larger batches).

Legal EASE — AI-powered Indian legal document analysis. Data generated by `scripts/benchmark_vector_dbs.py --runs 5` and `scripts/compare_answers.py`; full machine-readable results in `docs/benchmark_data.json`. Embeddings: law-ai/InLegalBERT (768-d, cosine). Both databases Apache-2.0.